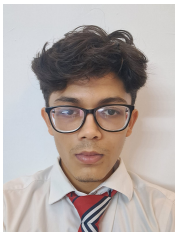
Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Orbital Collision Avoidance System

Team Name: *Team ID:*	*Cypher* *DSCPP-III-2026-T018*
Team member 1 (Team Lead) *Adamya Rawat* *Student ID: 2510011013* *2510011013@geu.ac.in*	
Team member 2 *Atiksh Sharma* *Student ID: 2510381135* *2510381135@geu.ac.in*	
Team member 3 *Agamya Saini* *Student ID: 2510385597* *2510385597@geu.ac.in*	
Team member 4 *Krishna Sehgal* *Student ID: 2510370197* *2510370197@geu.ac.in*	

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The Autonomous Orbital Collision Risk Assessment and Thruster Dispatch System can perform orbital collision risk assessments automatically and can make a quick decision for the required collision avoidance maneuver.The system will evaluate and classify conjunction alerts by considering multiple input features of the debris.
The system takes orbital conjunction alerts such as miss distance, relative velocity, Time of Closest Approach, covariance information, and collision risk.Machine Learning will be used to analyze this data and classify each conjunction alert into 4 risk levels : Nominal, Caution, High, or Critical.System uses data structures to process the incoming conjunction alerts and prioritize them based on the risk levels to ensure that the conjunction with the highest priority is processed first. The system comprise of various thrust engines like Chemical, Electric Ion, and Cold-Gas thrusters using a various factors such as the change in velocity(ΔV), fuel available, time to burn, and the time to the closest approach and decide which feasible collision-avoidance maneuver can take place. The project main aim is to create a autonomous orbital collision-risk assessment and thruster dispatch by implementing machine learning, data structures, object-oriented programming, data analysis, and event logging in a prototype system.

Updated Project Approach and Architecture

The main focus of Phase-II implementation is the development of the threat assessment and prioritization system. The system is implemented in C++ and Python used for the machine-learning pipeline. In the incoming orbital threat, information is represented using a C++ Threat data structure with parameter such as alert ID, miss distance, relative velocity, time to TCA, covariance trace, risk level and priority.
The incoming threats are first stored in a standard C++ FIFO Queue. Then the threat is transferred to a custom Circular Queue which acts as an internal processing buffer. The threat parameters are then passed to the Python machine-learning model using PyBind11. The model was trained using four features which are miss distance, relative velocity, time to TCA and covariance trace.
The Phase-I proposal considered Logistic Regression for risk classification.However, during Phase-II testing, we changed the model from Logistic Regression to Random Forest because the dataset is imbalanced and Random Forest gave better results in our experiments. The trained Random Forest model is then stored as model.pkl which is accessed through the Python prediction module.
The model predicts one of the four already defined risk levels which are Nominal, Caution, High or Critical. These risk levels are converted into numerical values 1 to 4 and are then inserted into a C++ Priority Queue. The highest-priority threat can then be identified for processing of the dispatch system.
We have used technology stack such as C++, Python, STL containers, PyBind11, pandas and scikit-learn.
The current architecture is: Threat Input -> Queue -> Circular Queue -> ML Model -> Risk Level -> Priority Queue -> Highest-Priority Threat
The thruster-dispatch and maneuver-management stages will be integrated in the next development stage.

Tasks Completed

Task Completed	Team Member
Literature Study, Existing Solutions and Research Gap Analysis — Studied existing orbital collision-avoidance approaches,studied relevant techniques and systems, identified the limitations of existing approaches and considered how our system could address them.	Krishna Sehga
Machine Learning Model Development — Prepared the machine-learning pipeline, selected the required features and risk categories, and trained and tested different classification models. Random Forest was selected and the trained model was saved for use in the next stages.	Agamya Saini
Overall System Design and Architecture, Dataset Analysis and Data Structure Implementation — Worked on the system workflow and architecture and analyzed the available orbital conjunction dataset,implemented the C++ Threat structure, FIFO Queue, custom Circular Queue, and Priority Queue for managing and prioritizing orbital threats.	Adamya Rawat Atiksh Sharma

Challenges/Roadblocks

During the phase 2 implementation of the project we faced several challenges. The first challenge was related to the dataset we have choosen , as the majority of the samples in the data were of Nominal risk category. This affected Logistic Regression's ability to classify the minority risk categories, so we tested different models and balancing techniques before choosing the final model.

Data preprocessing was also a challenge because the dataset needed to be modified according to the requirements of our system.We cannot use all of the data for the project implementation therefore several methods were used to extract only the necessary information, clean it, and form risk categories as required.

Another problem, which had to be resolved was related to the use of C. Initially, there was a plan to use C++ for thruster dispatch system and manually implemented C data structures for various tasks. However, this was a wrong decision, which created compatibility and integration issues and added unnecessary complexity to the project. Instead, it was decided to use the C++ STL data structures and implement the Circular Queue separately for the required task.

The integration of a Python ML model with C++ code using PyBind11 was another challenge. Initially, the C++ compiler was not compatible with the latest PyBind11 library requirements. However, this issue was resolved by finding a compiler, which would be compatible with Python and PyBind11.

Another challenge was connecting different data structures into one processing pipeline. The FIFO Queue, custom Circular Queue, and Priority Queue had to be integrated in a single flow, where each element would go through the process in a specific order. We tested each structure separately first and then tested the complete pipeline after integrating them.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
Thruster System Development: Development of C++ OOPs chemical, electric-ion, and cold-gas classes of thrusters and set the propulsion parameters of the classes.	Atiksh Sharma
Maneuver Feasibility and Thruster Selection: Implementation of various propulsion parameter such as ΔV, Burn-time, Fuel consumption, Available fuel, Thruster capability, Remaining TCA for selecting feasible collision avoidance maneuver.	Adamya Rawat
Maneuver History and Event Logging: Development of maneuver history management system using Stack and Linked List and implement the system logging mechanism for logging relevant events and maneuver decisions.	Agamya Saini
System Integration, Validation, and Testing: Integrate threat assessment, priority, thruster dispatch, maneuver, and logging modules and test the system.	Krishna Sehgal

Project Outcome/Deliverables

The project focuses on autonomous orbital collision-risk assessment and threat prioritization. The project uses a clean and preprocessed orbital conjunction dataset which contains parameters such as miss distance, relative velocity, time to TCA, and covariance trace.

A machine-learning risk classification pipeline was designed and evaluated and a Random Forest model was selected for the current implementation and saved using pickle module. The project delivers a C++ threat-management framework with Threat structure, FIFO Queue, custom Circular Queue, and Priority Queue.

PyBind11 module allows the communication between the C++ system and the Python-based ML model. The current prototype successfully handles incoming threats, predicts risk level (Nominal, Caution, High, Critical), assigns priority, and identifies the highest-priority threat.

The remaining deliverables to be completed are the thruster-dispatch module, maneuver feasibility analysis, maneuver history, event logging, and full system integration and validation.

Progress Overview

The project is currently in the core threat-assessment and prioritization stage. So far we have completed the literature study, system design, dataset collection, preprocessing of data, machine-learning pipeline, and C++ data-structure implementation. The Random Forest model was trained, evaluated and saved.Also the Python ML model has been successfully connected to the C++ application using PyBind11. The current system is able to take threat information, predict its risk level, assign a priority, and identify the highest-priority threat.

The remaining work is the thruster-dispatch and maneuver-management part of the system. This includes development of different thruster classes, performing maneuver feasibility calculations, checking fuel and burn time, maintaining maneuver history, and adding event history.After completing these modules, we will integrate the system and test it end-to-end.

Codebase Information

Github Repository Link-https://github.com/Divyaj-vanguard/Orbital-Collision-Avoidance-System.git

Current Commit include-
CircularQueue Implementation
Header file for custom Circular Queue
Test main file
pybind file
Branch-Main

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Dataset Validation	Pass	Dataset was cleaned, missing values were handled, relevant features were selected and risk categories were created.
ML Model Testing	Pass	Logistic Regression and Random Forest were evaluated. Random Forest achieved approximately 87 percent test accuracy and was selected for the current implementation.
Data Structure Testing	Pass	FIFO Queue, Circular Queue and Priority Queue were implemented and tested for threat insertion, processing, and prioritization.
C++–Python Integration Testing	Pass	PyBind11 integration was tested successfully, and the C++ application was able to call the Python ML model and receive risk predictions.
System Integration Testing	Pass	The current threat-assessment and prioritization pipeline has also been tested.

Deliverables Progress

Deliverables Progress
Literature study and research gap identification – completed
We have reviewed the research papers related to orbital collision avoidance measures and find out the key research gaps that are relevant to our project.
System design and architecture – completed
We have designed the general process of our system and also have a common understanding of how the system's elements will be processed and organized.
Dataset collection and preparation – completed
We have collect a conjunction dataset, preprocessed it,fill in the missing values,features were selected and risk categories were assigned.
Machine-learning model – completed
We have prepared the machine learning pipeline, trained and validated the model, and finally selected the Random Forest algorithm for the risk classification.
C++ threat management and data structures – completed
We have created a prototype of the Threat structure and FIFO Queue, Circular Queue, and Priority Queue data structures in C++.
C++–Python bridge – completed
We have connected the Python and C++ modules with the help of PyBind11 and successfully demonstrated the work flow.
Thruster and maneuver – in progress
The development of thruster classes, selection, and maneuver feasibility analysis are in progress.
Maneuver history and logging – in progress
The creation of the linked list or stack-based maneuver history and logging system event records in C++ are in progress.
Complete system integration and test – in progress
The final system test and integration will be performed after all the other deliverables are completed